Ejercicios de Clustering k-Means

Jan Żurek y Guido D'Inverno

04/2026

Contents

Ejercicio 1: Análisis Clustering de dataset 'estadisticas.txt' **1**
1.1. Análisis previos . . . 1
1.2. Selección del número de clústeres y clustering : 'PAM' . . . 3
1.3. Selección del número de clústeres y clustering : 'Manual' . . . 6
1.4. Selección del número de clústeres y clustering : 'NbClust' . . . 14

Ejercicio 2: Análisis Clustering de dataset 'ordenadores.txt' **19**
2.1. Estimación del número inicial de cluster . . . 19
2.3. Algorítmo K-medias . . . 25
2.4. PAM . . . 28
2.5. CLARA . . . 30

Ejercicio 1: Análisis Clustering de dataset 'estadisticas.txt'

1.1. Análisis previos

Antes de hacer un análisis de clúster, debemos verificar los datos recopilados y estandarizarlos para evitar que las variables medidas en diferentes escalas dominen el cálculo de distancias.

```
#Primero procedemos a leer el fichero de datos

datos_originales <-read.table("estadisticas.txt",sep="\t",header = T)

pander(head(datos_originales[, 1:5]))
```

nombre	posicion	minutosjugados	puntostotales	puntosmaximos
Dino Radoncic	Alero	403	88	10
William Magarity	Ala-pivot	438	150	16
Pere Tomas	Alero	268	89	13
Guillem Jou	Escolta	303	62	13
Eulis Baez	Ala-pivot	387	128	13
Rafa Martinez	Escolta	590	224	16

Carga correctamente el fichero de datos.

```
datos <- datos_originales[datos_originales$minutosjugados >= 100, ]

# Usamos gsub para sustituir la coma por un punto antes de la conversión
datos$tirosd3 <- as.numeric(gsub(",", ".", datos$tirosd3))
```

```r
datos$tirosd2 <- as.numeric(gsub(",", ".", datos$tirosd2))
datos$tirosde1 <- as.numeric(gsub(",", ".", datos$tirosde1))


# Verificación de valores faltantes
print(sum(is.na(datos)))
```

```
## [1] 0
```

```r
datos_cluster <- datos[, -(1:3)]

datos_cluster[] <- lapply(datos_cluster, as.numeric)

str(datos_cluster)
```

```
## 'data.frame':    149 obs. of  19 variables:
##  $ puntostotales      : num  88 150 89 62 128 224 167 128 189 46 ...
##  $ puntosmaximos      : num  10 16 13 13 13 16 19 14 15 11 ...
##  $ tirosd3convertidos : num  9 25 11 12 17 34 6 10 17 6 ...
##  $ tirosde3intentados : num  26 76 34 39 40 86 23 54 58 13 ...
##  $ tirosd3            : num  34.6 32.9 32.4 30.8 42.5 39.5 26.1 18.5 29.3 46.2 ...
##  $ tirosde2convertidos: num  26 31 20 8 29 35 65 32 53 11 ...
##  $ tirosde2intentados : num  61 53 28 15 50 79 115 77 96 25 ...
##  $ tirosd2            : num  42.6 58.5 71.4 53.3 58 44.3 56.5 41.6 55.2 44 ...
##  $ tirosde1convertidos: num  9 13 16 10 19 52 19 34 32 6 ...
##  $ tirosde1intentados : num  17 15 20 14 36 62 36 46 44 7 ...
##  $ tirosde1           : num  52.9 86.7 80 71.4 52.8 83.9 52.8 73.9 72.7 85.7 ...
##  $ rebotesofensivos   : num  30 20 14 12 17 4 27 12 23 3 ...
##  $ rebotesdefensivos  : num  60 50 43 28 38 45 44 32 78 6 ...
##  $ rebotestotales     : num  90 70 57 40 55 49 71 44 101 9 ...
##  $ asistencias        : num  9 8 12 11 26 53 13 137 109 15 ...
##  $ recuperaciones     : num  11 8 7 6 7 10 11 19 25 6 ...
##  $ perdidas           : num  18 16 7 10 15 45 21 46 55 11 ...
##  $ taponesafavor      : num  0 9 0 5 4 0 4 0 10 3 ...
##  $ valoracion         : num  84 133 118 62 152 210 144 195 267 36 ...
```

```
  No hay valores que falten
```

Hemos corregido los separadores y convertido en variables numéricas aquellas que antes eran categóricas.

Hemos eliminado las variables que no nos interesan para el análisis de conglomerados.

Hemos eliminado a los jugadores con menos de 100 minutos jugados; ahora tenemos 149 observaciones.

Normalización de los datos

```r
datoz <- scale(datos_cluster, center = TRUE, scale = TRUE)

# Crea un dataframe di riepilogo
riepilogo <- data.frame(
  Media = round(colMeans(datoz), 3),
  Dev_Std = apply(datoz, 2, sd)
)

pander(riepilogo)
```

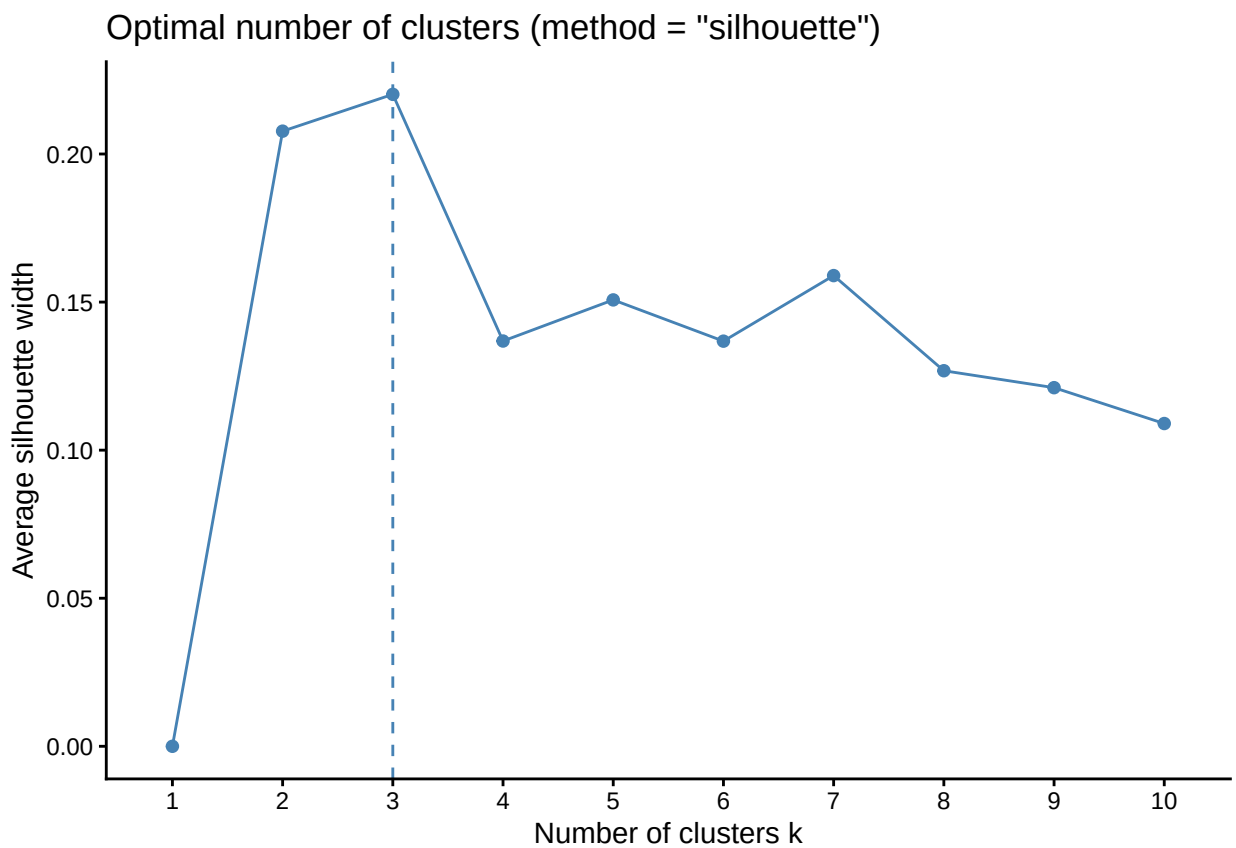
	Media	Dev_Std
puntostotales	0	1
puntosmaximos	0	1
tirosd3convertidos	0	1
tirosde3intentados	0	1
tirosd3	0	1
tirosde2convertidos	0	1
tirosde2intentados	0	1
tirosd2	0	1
tirosde1convertidos	0	1
tirosde1intentados	0	1
tirosde1	0	1
rebotesofensivos	0	1
rebotesdefensivos	0	1
rebotestotales	0	1
asistencias	0	1
recuperaciones	0	1
perdidas	0	1
taponesafavor	0	1
valoracion	0	1

Datos correctamente estandarizados (media=0, dev.st=1) para que todos tengan el mismo "peso".

1.2. Selección del número de clústeres y clustering : 'PAM'

Analicemos varios métodos para calcular el valor óptimo de k: empecemos por el 'PAM'

```
rownames(datoz) <- datos$nombre

fviz_nbclust(datoz, pam, method = "silhouette") +
  theme_classic()
```




```r
pam.res <- pam(datoz, 3)

print(pam.res$medoids)
```

```
##                puntostotales puntosmaximos tirosd3convertidos
## Guillem Jou       -1.1322934    -0.7952145         -0.4677394
## Jeremy Senglin     0.7685673    -0.1441872          0.4176548
## Tyson Perez       -0.4298014    -0.6324577         -0.7839516
##                tirosde3intentados      tirosd3 tirosde2convertidos
## Guillem Jou            -0.3663280  0.008163031         -1.18735210
## Jeremy Senglin          0.5916972  0.182917578          0.66591222
## Tyson Perez            -0.6611049 -0.334355881          0.06003734
##                tirosde2intentados     tirosd2 tirosde1convertidos
## Guillem Jou           -1.31108537  0.05920057          -0.8271322
## Jeremy Senglin         1.01758234 -0.46725909           0.6850481
## Tyson Perez           -0.05061386  0.57491616          -0.3098074
##                tirosde1intentados   tirosde1 rebotesofensivos rebotesdefensivos
## Guillem Jou            -0.8920980 -0.2748751       -0.4482097        -0.7192475
## Jeremy Senglin          0.4306837  1.1810826       -0.4482097        -0.1885383
## Tyson Perez            -0.2476659 -0.4037880        1.4118807         0.5898352
##                rebotestotales asistencias recuperaciones   perdidas
## Guillem Jou        -0.6475058  -0.7282861     -0.9838011 -1.0760880
## Jeremy Senglin     -0.2981153   0.9100077      0.8732897  1.3413827
## Tyson Perez         0.9363978  -0.7878604     -0.4885769 -0.3573805
##                taponesafavor valoracion
## Guillem Jou      -0.02497868 -1.0953548
```

```
## Jeremy Senglin   -0.64528263  0.3798408
## Tyson Perez       0.84344684 -0.1501696
```

El PAM nos sugiere k = 3 grupos.

Interpretación del «jugador típico» (el medoide) en los 3 grupos:

- *Clúster 1 (ID 4 - Guillem Jou -)* : Todos los valores son muy negativos; son jugadores que anotan poco, juegan poco y tienen un impacto bajo en casi todas las estadísticas.
- *Clúster 2 (ID 101 - Jeremy Senglin -)*: Tienen valores altos en asistencias (0,91), recuperaciones (0,87) y tiros de 3 (porcentaje de 3). Probablemente sean bases o bases-escoltas que dirigen el juego, anotan desde fuera pero tienen dificultades en el rebote ofensivo (-0,44).
- *Clúster 3 (ID 92 - Tyson Pérez-)* : Dominan en rebotes totales (0,93) y tapones a favor (0,84), pero tienen promedios muy bajos en tiros desde fuera y en asistencias. Estos son los clásicos pívots o aleros altos que trabajan bajo el aro.

```r
pander(pam.res$clusinfo)
```

size	max_diss	av_diss	diameter	separation
58	4.571	2.761	7.943	1.029
54	10.03	3.757	13.18	1.796
37	8.319	3.779	10.19	1.029

Aquí, en cambio, podemos observar lo separados que están los grupos.

- El clúster que mejor se distingue por sus características es el 2, sin embargo, es el más variado (menos compacto) y presenta valores más altos de max_diss, av_diss y diameter.
- El clúster más 'compacto' es el 1, que presenta valores más bajos de max_diss, av_diss y diameter.

```r
p <- fviz_cluster(pam.res,
             data = datoz,
             palette = c("#00AFBB", "#FC4E07", "#228B22"),
             ellipse.type = "t",
             repel = TRUE,
             labelsize = 7,
             pointsize = 1.5,
             ggtheme = theme_minimal()) +
     labs(title = "Análisis de clústeres de jugadores de la ACB")

# Visualizza
p
```

```
options(repr.plot.width = 15, repr.plot.height = 15)
```

Podemos observar que, cuando k = 3, gráficamente algunos jugadores son auténticas 'anomalías' que no encajan en los grupos.

Además, podemos observar que los gráficos se superponen, por lo que no permiten distinguir claramente los grupos en función de sus características.

1.3. Selección del número de clústeres y clustering : 'Manual'

Ahora veamos otro método para calcular el valor óptimo de k: el método 'Manual'

```
n <- dim(datoz)[1]
k.min <- 2
k.max <- 20
```

```r
wss <- numeric()
set.seed(12345)
wss[1] <- (n - 1) * sum(apply(datoz, 2, var))

for(k in k.min:k.max){
  wss[k] <- kmeans(datoz, centers = k)$tot.withinss
}
wss
```

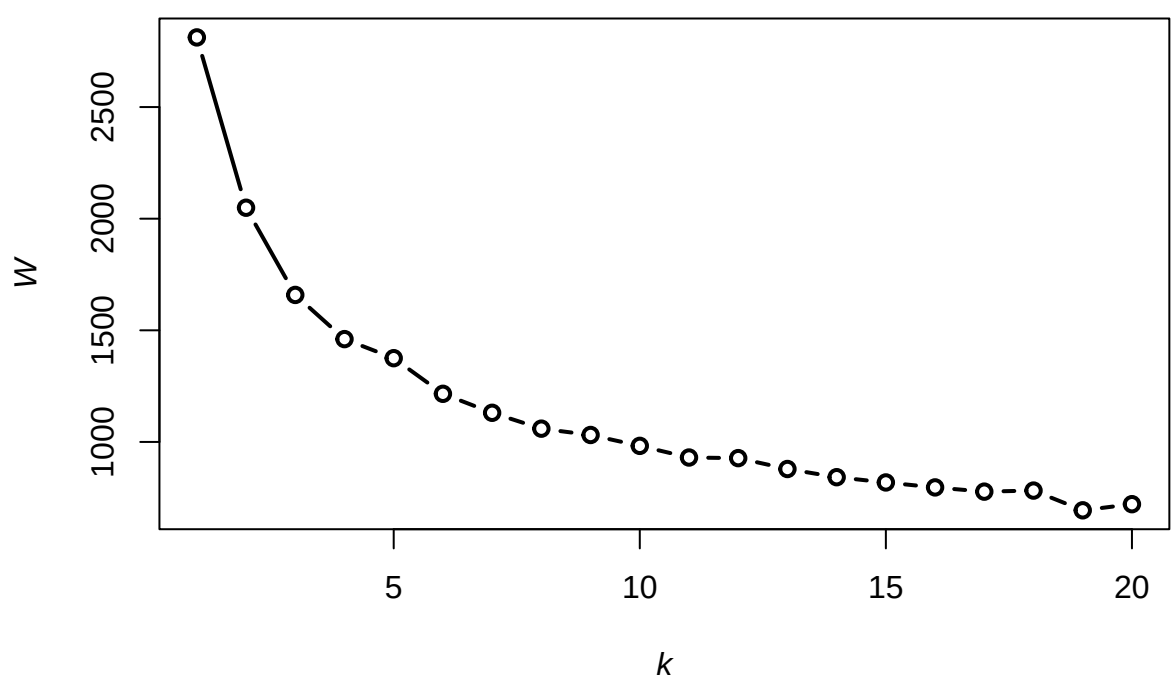
```
##  [1] 2812.0000 2049.1299 1658.5353 1460.6454 1375.2478 1215.5921 1130.6088
##  [8] 1059.4903 1031.2361  982.4370  930.3351  927.6019  878.8395  842.1540
## [15]  818.5781  796.4137  777.4097  782.2184  693.8321  721.2401
```

```r
diff(wss)
```

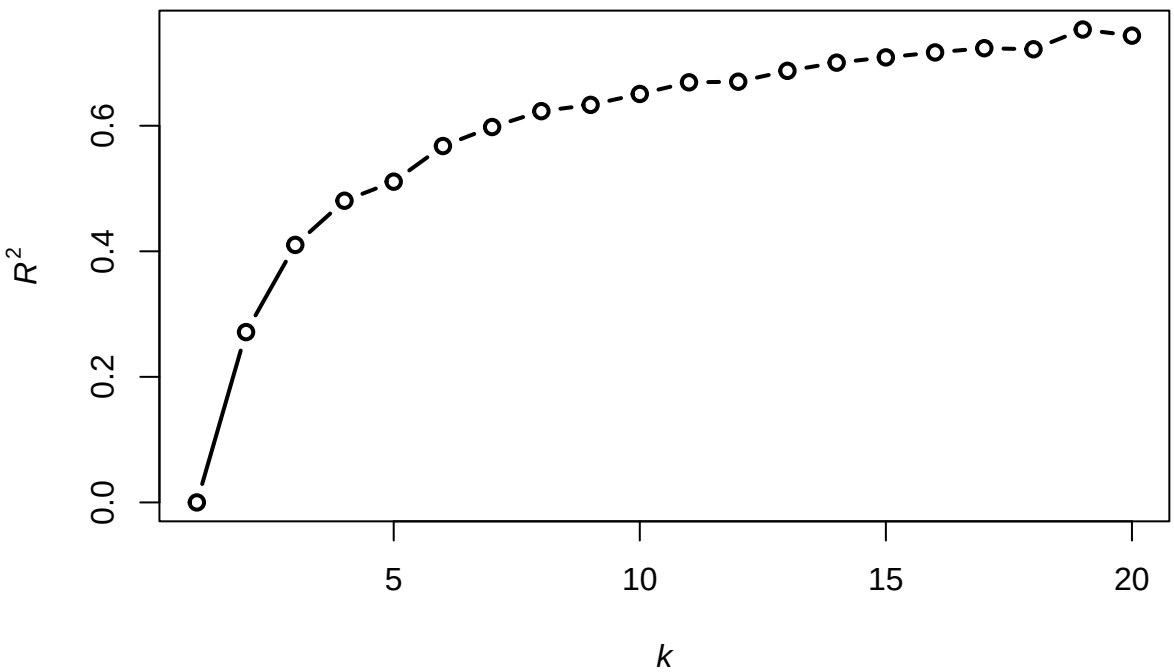
```
##  [1] -762.870148 -390.594515 -197.889982  -85.397551 -159.655656  -84.983392
##  [7]  -71.118410  -28.254286  -48.799090  -52.101872   -2.733168  -48.762458
## [13]  -36.685421  -23.575957  -22.164356  -19.004005    4.808654  -88.386293
## [19]   27.408020
```

Los resultados indican que el número ideal de clústeres es de 3 a 5, ya que a partir de ahí la reducción de la 'distancia interna' (WSS) se vuelve mucho menor, por lo que añadir más clústeres ya no ayuda a explicar mejor los datos, sino que solo complica el modelo.

```r
plot(1:k.max, wss,
     xlab = expression(italic(k)),
     ylab = expression(italic(W)),
     type = "b", lwd = 2)
```



```r
R2 <- 1 - wss/wss[1]
plot(1:k.max, R2,
     xlab = expression(italic(k)),
     ylab = expression(italic(R^2)),
     type = "b", lwd = 2)
```

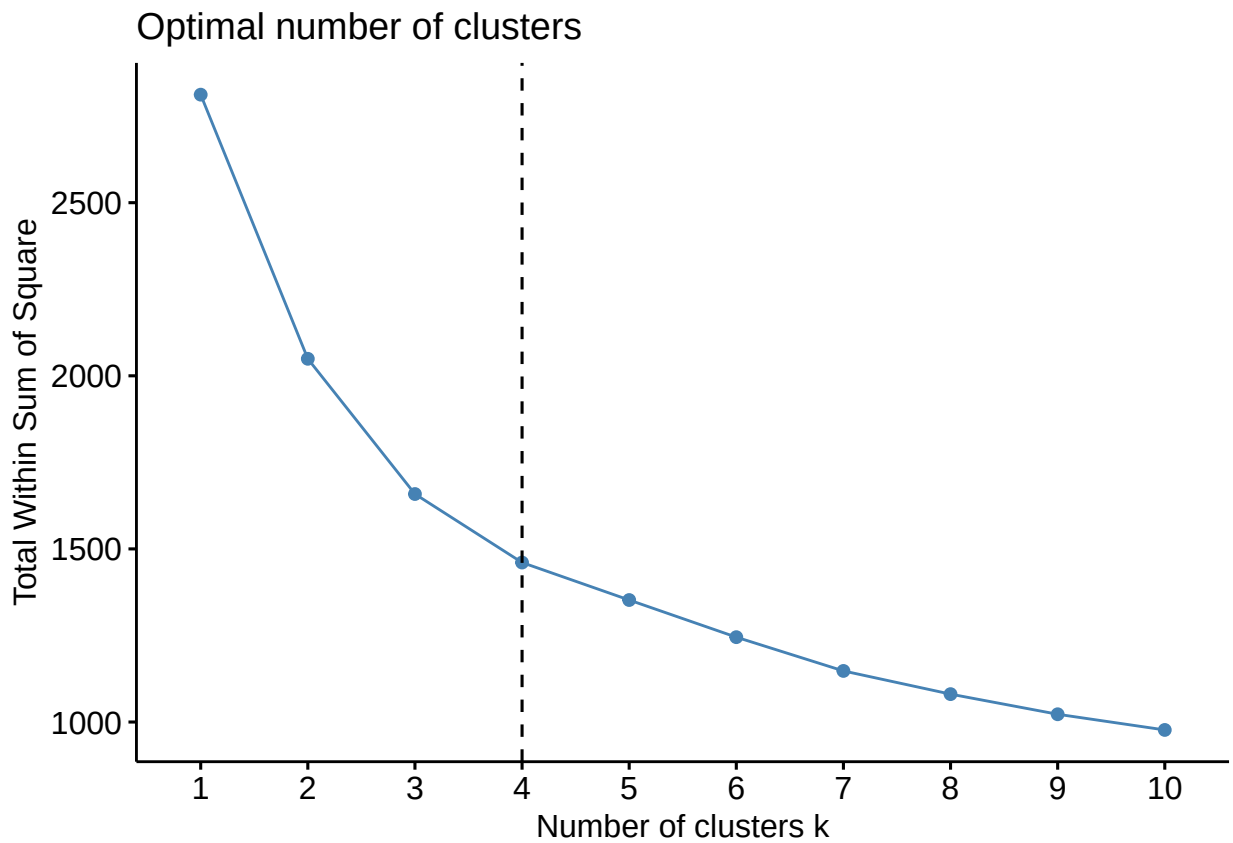


También gráficamente se confirma que el valor óptimo de k está entre 3 y 5 clústeres.

Sin embargo, como ya hemos visto qué pasaría con k = 3, comprobemos los otros dos casos.

En este punto, creamos un clúster comparando k = 4 y k = 5

```r
fviz_nbclust(datoz,kmeans, method = "wss")+ geom_vline(xintercept = 4, linetype = 2)
```




```r
set.seed(12345)
res.kmeans.4 <- kmeans(datoz, centers = 4, nstart = 50, iter.max = 1000)

print(res.kmeans.4$size)
```

```
## [1] 24 55 59 11
```

```r
print(res.kmeans.4$withinss)
```

```
## [1] 327.2478 394.6387 491.9779 246.6890
```

```r
round(res.kmeans.4$centers, 2)
```

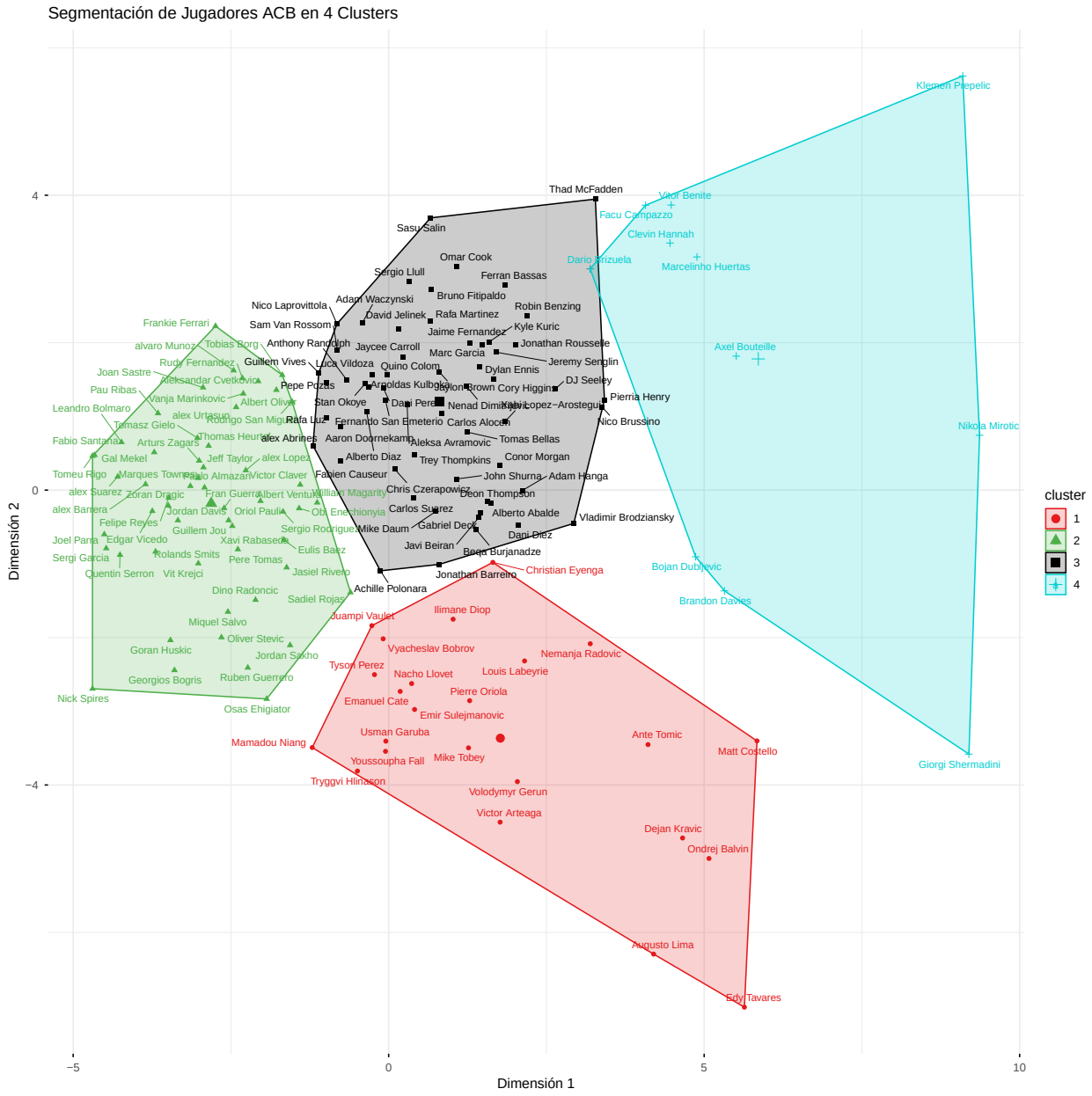
```
##   puntostotales puntosmaximos tirosd3convertidos tirosde3intentados tirosd3
## 1          0.16          0.15              -0.91              -0.94   -1.01
## 2         -0.92         -0.68              -0.54              -0.55   -0.03
## 3          0.38          0.27               0.63               0.67    0.38
## 4          2.23          1.59               1.28               1.20    0.30
##   tirosde2convertidos tirosde2intentados tirosd2 tirosde1convertidos
## 1                1.03               0.85    0.86                0.05
## 2               -0.83              -0.86   -0.26               -0.69
## 3                0.04               0.13   -0.14                0.17
## 4                1.69               1.79    0.17                2.41
##   tirosde1intentados tirosde1 rebotesofensivos rebotesdefensivos rebotestotales
## 1               0.36    -0.94             1.58              1.10           1.34
## 2              -0.73    -0.16            -0.50             -0.72          -0.67
## 3               0.12     0.37            -0.23              0.07          -0.04
## 4               2.23     0.85             0.30              0.80           0.64
```

```
##   asistencias recuperaciones perdidas taponesafavor valoracion
## 1       -0.52          -0.06     0.03          1.47       0.57
## 2       -0.48          -0.73    -0.79         -0.33      -0.92
## 3        0.46           0.53     0.39         -0.28       0.25
## 4        1.07           0.94     1.82         -0.06       2.01
```

El análisis ha dividido la liga en : Especialistas defensivos (1), Reservas (2), Jugadores de equipo (3) y Superestrellas (4).

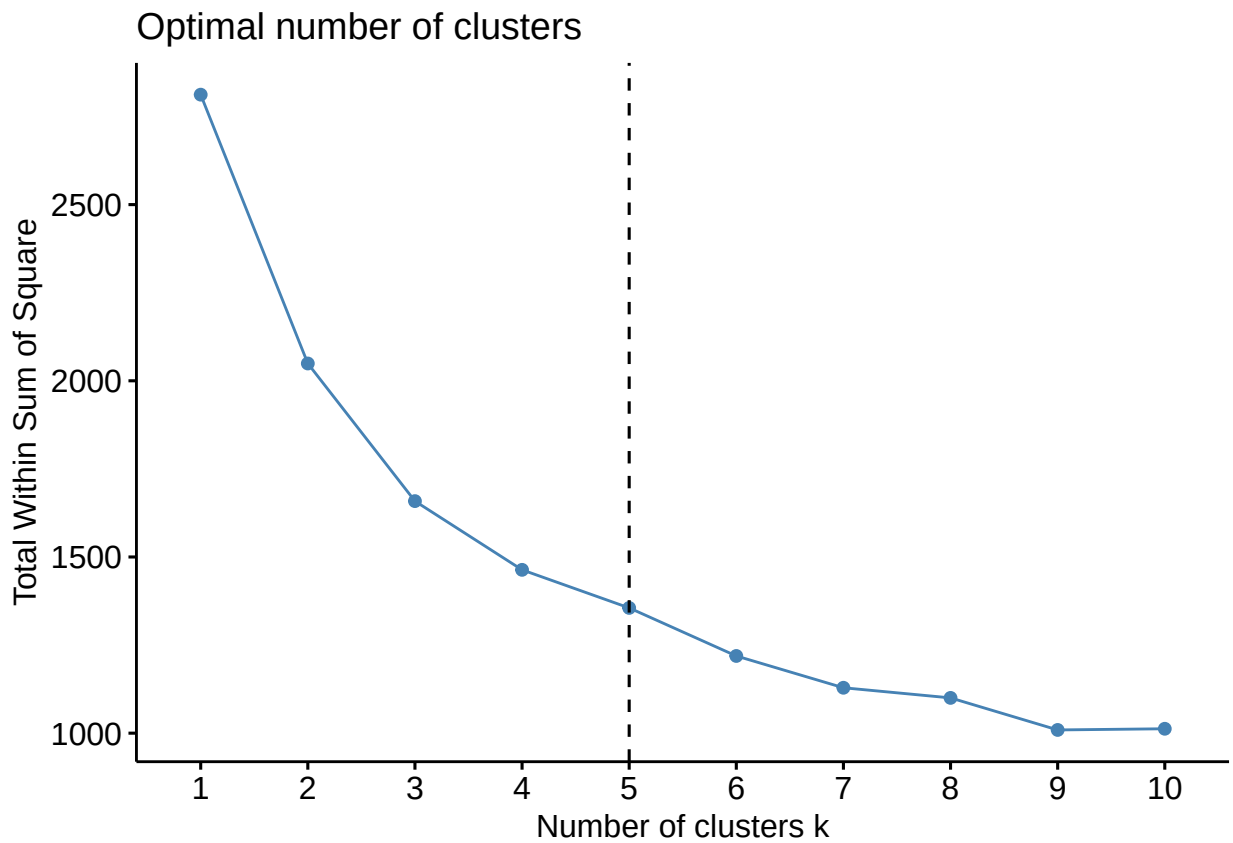
La división en cuatro grupos agrega un nivel jerárquico basado en el valor absoluto, separando claramente a las 'superestrellas'.

```r
colori_unici <- c("#E41A1C", "#4DAF4A",
                      "#000000", "#00CED1")

# Crear el gráfico asegurando que pasamos los nombres
p4 <- fviz_cluster(res.kmeans.4,
             data = datoz,           # El dataset con rownames actualizados
             palette = colori_unici,
             ellipse.type = "convex",
             ellipse.alpha = 0.2,
             repel = TRUE,           # Evita que se solapen los nombres
             labelsize = 8,
             show.clust.cent = TRUE, # Útil para ver el "líder" de cada grupo
             ggtheme = theme_minimal()) +
  labs(title = "Segmentación de Jugadores ACB en 4 Clusters",
       x = "Dimensión 1",
       y = "Dimensión 2")

# Ajustar dimensiones y mostrar
options(repr.plot.width = 15, repr.plot.height = 15)
print(p4)
```



Ahora lo comprobamos con k = 5

```
fviz_nbclust(datoz,kmeans, method = "wss")+ geom_vline(xintercept = 5, linetype = 2)
```




```r
set.seed(12345)
res.kmeans.5 <- kmeans(datoz, centers = 5, nstart = 50, iter.max = 1000)

print(res.kmeans.5$size)
```

```
## [1]  9  9 55 29 47
```

```r
print(res.kmeans.5$withinss)
```

```
## [1] 161.7979 136.2026 387.2758 257.6634 358.3596
```

```r
round(res.kmeans.5$centers, 2)
```

```
##   puntostotales puntosmaximos tirosd3convertidos tirosde3intentados tirosd3
## 1          2.31          1.65               2.06               2.01    0.50
## 2          1.23          0.72              -0.78              -0.86   -1.24
## 3         -0.92         -0.66              -0.51              -0.52   -0.02
## 4          0.09          0.11              -0.43              -0.51   -0.21
## 5          0.34          0.25               0.62               0.71    0.30
##   tirosde2convertidos tirosde2intentados tirosd2 tirosde1convertidos
## 1                1.20               1.37    0.00                2.33
## 2                2.31               2.09    0.88                1.04
## 3               -0.83              -0.87   -0.24               -0.70
## 4                0.53               0.40    0.76               -0.01
## 5               -0.03               0.11   -0.36                0.18
##   tirosde1intentados tirosde1 rebotesofensivos rebotesdefensivos rebotestotales
## 1               2.09     0.93            -0.31              0.26           0.05
## 2               1.37    -0.43             2.41              2.31           2.46
```

```
## 3              -0.74    -0.17            -0.54             -0.73          -0.69
## 4               0.16    -0.58             1.03              0.69           0.86
## 5               0.11     0.46            -0.40             -0.06          -0.20
##   asistencias recuperaciones perdidas taponesafavor valoracion
## 1        1.40           1.16     1.79         -0.30       1.80
## 2       -0.11           0.75     0.89          2.26       1.96
## 3       -0.49          -0.73    -0.80         -0.33      -0.92
## 4       -0.52          -0.23    -0.21          0.59       0.32
## 5        0.64           0.63     0.55         -0.35       0.16
```

Pasar de 4 a 5 grupos permite al algoritmo comprender que los jugadores de élite no son todos iguales, separando claramente a las 'estrellas exteriores'(cluster 1), que crean juego y anotan desde lejos, de los 'estrellas internas'(cluster 2), que dominan físicamente bajo el aro.

```r
colori_unici <- c("#E41A1C", "#4DAF4A",
                      "#000000", "#00CED1","#A65628")

# Crear el gráfico asegurando que pasamos los nombres
p5 <- fviz_cluster(res.kmeans.5,
             data = datoz,           # El dataset con rownames actualizados
             palette = colori_unici,
             ellipse.type = "convex",
             ellipse.alpha = 0.2,
             repel = TRUE,           # Evita que se solapen los nombres
             labelsize = 8,
             show.clust.cent = TRUE, # Útil para ver el "líder" de cada grupo
             ggtheme = theme_minimal()) +
  labs(title = "Segmentación de Jugadores ACB en 5 Clusters",
       x = "Dimensión 1",
       y = "Dimensión 2")

# Ajustar dimensiones y mostrar
options(repr.plot.width = 15, repr.plot.height = 15)
print(p5)
```

1.4. Selección del número de clústeres y clustering : 'NbClust'

Otro método para encontrar el valor óptimo de k es 'NbClust', que calcula decenas de índices estadísticos al mismo tiempo.

```
res.kmeans.NbClust <- NbClust(datoz,distance="euclidean",
min.nc= 2, max.nc= 20,
method= "kmeans",index="kl")


pander(round(res.kmeans.NbClust$Best.nc,0))
```

Number_clusters	Value_Index
11	3

```r
print(round(res.kmeans.NbClust$Best.partition,0))
```

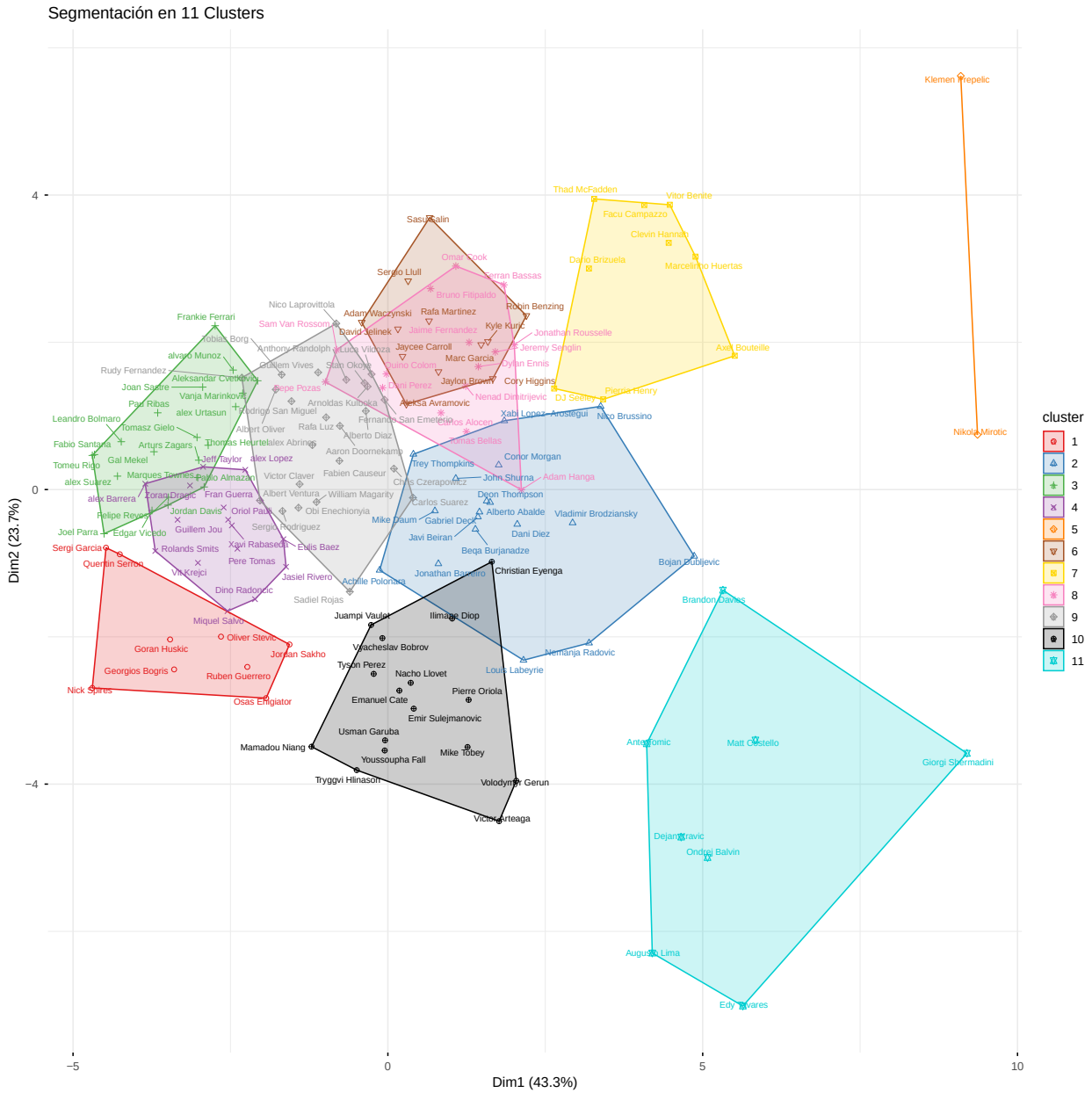
```
##         Dino Radoncic      William Magarity            Pere Tomas
##                     4                     9                     4
##           Guillem Jou            Eulis Baez         Rafa Martinez
##                     4                     4                     6
##         Juampi Vaulet            Dani Perez            Adam Hanga
##                    10                     8                     8
##       Leandro Bolmaro         Rolands Smits        Thomas Heurtel
##                     3                     4                     3
##         Pierre Oriola          alex Abrines          Cory Higgins
##                    10                     9                     6
##            Kyle Kuric        Nikola Mirotic        Brandon Davies
##                     6                     5                    11
##         Victor Claver     Jonathan Barreiro             DJ Seeley
##                     9                     2                     7
##         Nico Brussino         Robin Benzing            Vit Krejci
##                     2                     6                     4
##    Rodrigo San Miguel           Dylan Ennis      Tryggvi Hlinason
##                     9                     8                    10
##             Pau Ribas  Xabi Lopez-Arostegui          Conor Morgan
##                     3                     2                     2
##  Vladimir Brodziansky        Albert Ventura         Ferran Bassas
##                     2                     9                     8
##         Arturs Zagars            Joel Parra            Ante Tomic
##                     3                     3                    11
##    Nenad Dimitrijevic           Tobias Borg         Mamadou Niang
##                     8                     9                    10
##         Pablo Almazan           Nick Spires       Obi Enechionyia
##                     3                     1                     9
##       Frankie Ferrari            Stan Okoye           John Shurna
##                     3                     9                     2
##         Fabio Santana       Beqa Burjanadze         Matt Costello
##                     3                     2                    11
##           Javi Beiran          Dejan Kravic          Vitor Benite
##                     2                    11                     7
##          alex Barrera          Miquel Salvo         Thad McFadden
##                     4                     4                     7
##         Jasiel Rivero          Jordan Sakho             Omar Cook
##                     4                     1                     8
##         Xavi Rabaseda          Goran Huskic       Bruno Fitipaldo
##                     4                     1                     8
##    Marcelinho Huertas            Sasu Salin             Dani Diez
##                     7                     6                     2
##      Sergio Rodriguez     Giorgi Shermadini     Emir Sulejmanovic
##                     9                    11                    10
##            alex Lopez           Fran Guerra      Aaron Doornekamp
##                     4                     4                     9
##       Georgios Bogris          alvaro Munoz         Albert Oliver
##                     1                     3                     9
```

```
##             Mike Daum     Chris Czerapowicz            Pepe Pozas
##                     2                     9                     8
##           alex Suarez           Tyson Perez           Oriol Pauli
##                     3                    10                     4
##          Nacho Llovet         Clevin Hannah          Tomasz Gielo
##                    10                     7                     3
##          Sergi Garcia         David Jelinek        Jeremy Senglin
##                     1                     6                     8
##      Aleksa Avramovic          Edgar Vicedo  Aleksandar Cvetkovic
##                     6                     3                     3
##        Victor Arteaga    Jonathan Rousselle          Jaylon Brown
##                    10                     8                     6
##         Ondrej Balvin        Quentin Serron      Arnoldas Kulboka
##                    11                     1                     9
##            Tomeu Rigo      Anthony Randolph        Rudy Fernandez
##                     3                     9                     9
##        Alberto Abalde         Facu Campazzo     Nico Laprovittola
##                     2                     7                     9
##          Felipe Reyes        Jaycee Carroll           Edy Tavares
##                     3                     6                    11
##          Sergio Llull        Trey Thompkins           Jeff Taylor
##                     6                     2                     4
##         Carlos Alocen          Usman Garuba        Fabien Causeur
##                     8                    10                     9
##          Gabriel Deck          Luca Vildoza         Pierria Henry
##                     2                     9                     7
##          Ilimane Diop       Youssoupha Fall          Zoran Dragic
##                    10                    10                     4
##      Achille Polonara          Jordan Davis        Marques Townes
##                     2                     3                     3
##              Rafa Luz       Nemanja Radovic          Emanuel Cate
##                     9                     2                    10
##          Augusto Lima          Sadiel Rojas         Deon Thompson
##                    11                     9                     2
##       Jaime Fernandez        Dario Brizuela          Alberto Diaz
##                     8                     7                     9
##        Adam Waczynski        Ruben Guerrero         Carlos Suarez
##                     6                     1                     9
##       Volodymyr Gerun        Axel Bouteille             Gal Mekel
##                    10                     7                     3
##          alex Urtasun        Osas Ehigiator           Marc Garcia
##                     3                     1                     6
##      Christian Eyenga         Oliver Stevic          Tomas Bellas
##                    10                     1                     8
##     Vyacheslav Bobrov           Quino Colom      Vanja Marinkovic
##                    10                     8                     3
##       Klemen Prepelic        Louis Labeyrie        Sam Van Rossom
##                     5                     2                     8
##            Mike Tobey       Bojan Dubljevic         Guillem Vives
##                    10                     2                     9
## Fernando San Emeterio           Joan Sastre
##                     9                     3
```

NbClust es mucho más exigente, intenta crear grupos muy específicos; por eso el número de clústeres es tan

elevado.

```
colori_unici <- c("#E41A1C", "#377EB8", "#4DAF4A", "#984EA3", "#FF7F00",
                  "#A65628", "gold", "#F781BF", "#999999", "#000000", "#00CED1")


pp <-  fviz_cluster(list(data = datoz, cluster = res.kmeans.NbClust$Best.partition),
             palette = colori_unici,
             ellipse.type = "convex",
             ellipse.alpha = 0.2,
             repel = TRUE,
             labelsize = 7,
             show.clust.cent = FALSE,
             ggtheme = theme_minimal()) +
  labs(title = "Segmentación en 11 Clusters" )

# Visualizza
pp
```




```
options(repr.plot.width = 15, repr.plot.height = 15)
```

Con k = 11 obtenemos una clasificación detallada de las funciones modernas de la ACB, pero aumenta la complejidad y la dificultad de interpretación de la agrupación.

Conclusion

Al final elegimos k = 5 porque es el equilibrio perfecto entre simplicidad estadística y realidad táctica, ya que es el único modelo capaz de separar a las superestrellas en dos perfiles distintos (bases creadores vs. centros dominantes).

Esta división transforma los datos en una verdadera pizarra de baloncesto, permitiendo identificar no solo quién destaca, sino también qué rol específico desempeña cada jugador en la pista.

Ejercicio 2: Análisis Clustering de dataset 'ordenadores.txt'

El fichero `ordenadores.txt` contiene las características de 6259 ordenadores personales. Se pide realizar un análisis cluster de k-medias para clasificarlos.

Cargamos el archivo como un marco de datos, mostramos su aspecto y normalizamos los datos para que ninguna variable domine todo el conjunto de datos.

```
ordenadores <- data.frame(read.table("ordenadores.txt", sep="\t", header = T))

pander(head(ordenadores))
```

price	speed	hd	ram	screen	ads	trend
1499	25	80	4	14	94	1
1795	33	85	2	14	94	1
1595	25	170	4	15	94	1
1849	25	170	8	14	94	1
3295	33	340	16	14	94	1
3695	66	340	16	14	94	1

```
ordenadores <- as.data.frame(scale(ordenadores))
```

Comprobamos si el conjunto de datos está completo, es decir, si hay datos que faltan (`NA`).

```
sum(is.na(ordenadores))
```

```
## [1] 0
```

2.1. Estimación del número inicial de cluster

Empezaremos por determinar el número óptimo de clústeres para los siguientes métodos:

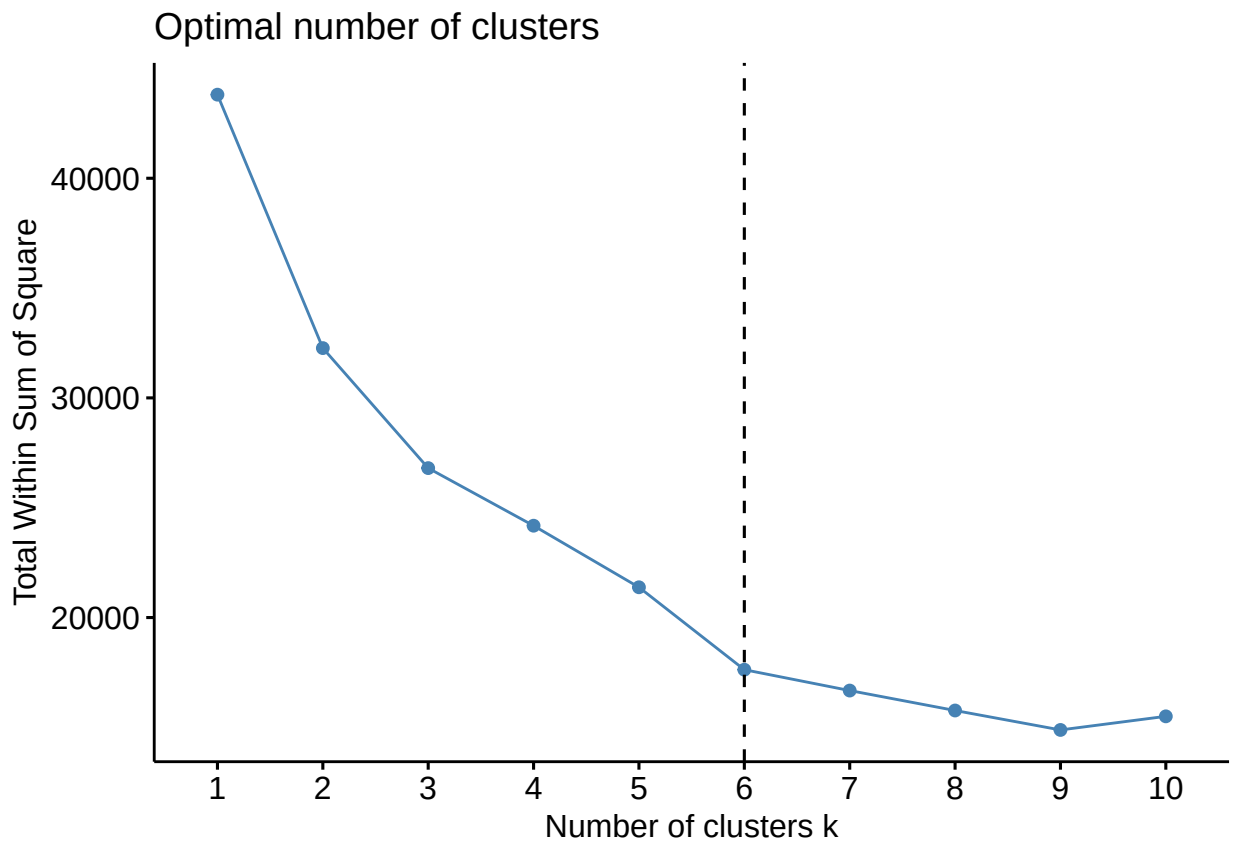
1. K-means
2. PAM
3. CLARA

2.1. K-medias - número inicial de cluster

WSS

Empecemos por el método más sencillo para determinar el número óptimo de clústeres, es decir, el método `wss` (*within-cluster sum of squares*), que mide la cohesión interna de los clústeres, es decir, la varianza del clúster.

```
set.seed(2137)
fviz_nbclust(ordenadores, kmeans, method = "wss")+
  geom_vline(xintercept = 6, linetype = 2)
```



Se observa un claro pliegue (el «codo» de la gráfica) en el punto 6, por lo que, según el método `wss`, el número óptimo de clústeres es **6**.

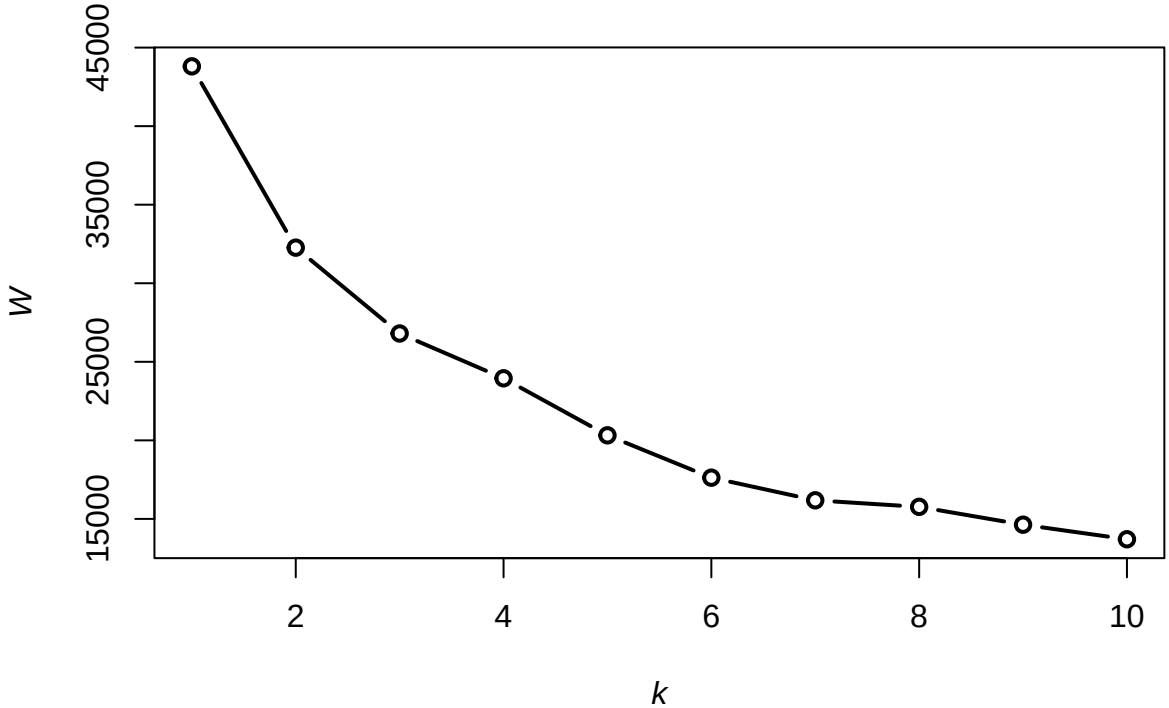
WSS - Manual

La segunda forma, análoga a la anterior, consiste en generar manualmente el gráfico anterior, lo que, como era de esperar, arroja resultados similares, es decir, sugiere **6** o **7** clústeres.

```
n <- dim(ordenadores)[1]
k.min <- 2
k.max <- 10
wss <- numeric ()
set.seed(2137)
wss [1] <- (n - 1)*sum(apply(ordenadores, 2, var))
for (k in k.min:k.max){
  wss[k] <- kmeans(ordenadores, centers = k)$tot.withinss
}
wss
```

```
##  [1] 43806.00 32266.74 26803.27 23950.47 20319.41 17624.92 16184.91 15768.09
##  [9] 14632.61 13706.22
```

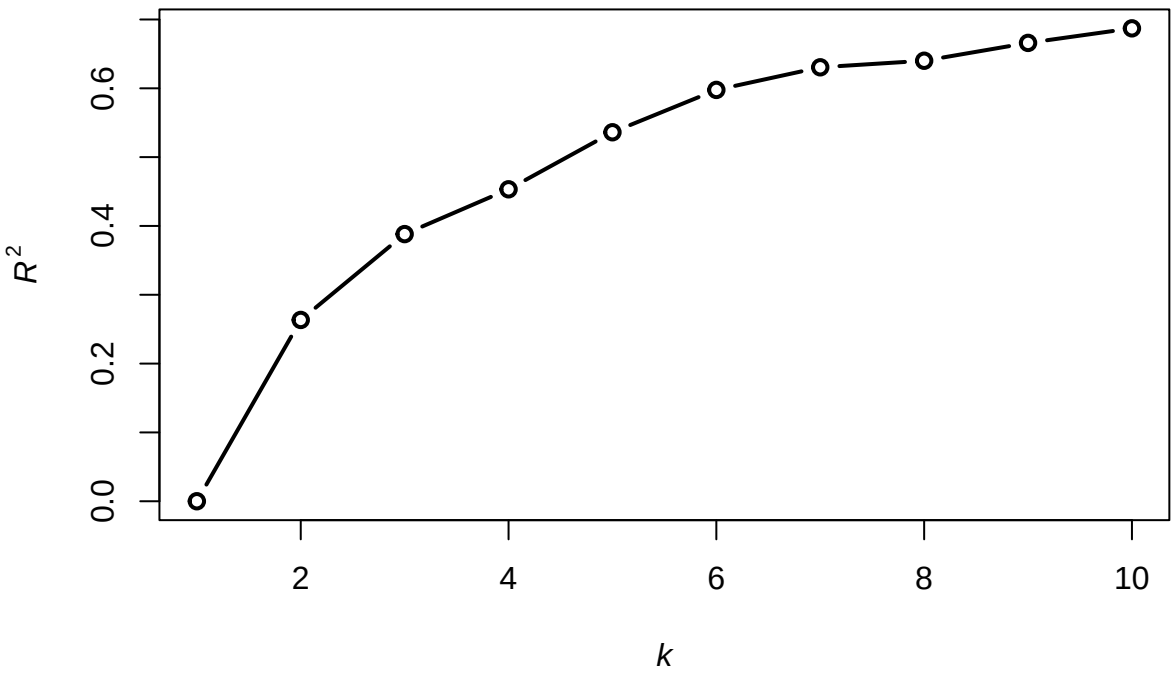
```
plot (1:k.max , wss ,
      xlab = expression(italic(k)),
      ylab = expression(italic(W)),
      type = "b", lwd = 2)
```




```r
diff(wss)
```

```
## [1] -11539.2634  -5463.4692  -2852.7927  -3631.0675  -2694.4892  -1440.0127
## [7]   -416.8123  -1135.4838   -926.3919
```

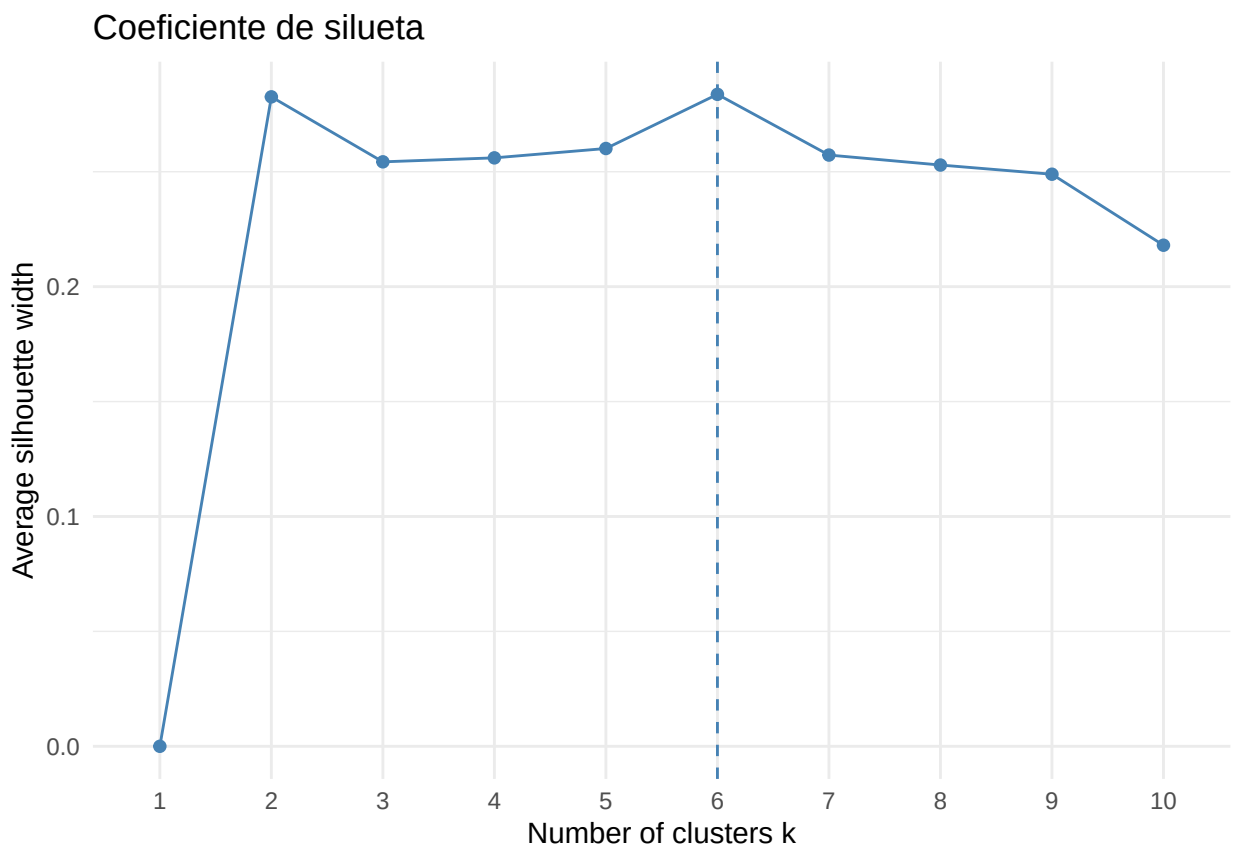
```r
R2 <- 1 - wss/wss [1]
plot (1:k.max , R2 ,
      xlab = expression(italic(k)),
      ylab = expression(italic(R^2)),
      type = "b", lwd = 2)
```



Silhouette

Un método más completo es el método «silhouette», que mide simultáneamente la cohesión interna y externa de cada uno de los clústeres. Para cada punto, calcula el coeficiente $s(i)$, que evalúa en qué medida ese punto pertenece a su propio clúster y a qué distancia se encuentra del clúster vecino más cercano.

```
set.seed(420)
fviz_nbclust(ordenadores, kmeans, method = "silhouette", k.max = 10) +
  theme_minimal() +
  ggtitle("Coeficiente de silueta")
```



Vemos que el método `silhouette` también sugiere elegir **6 clústeres** para el método `k-means`, por lo que continuaremos el análisis con ese número.

2.2. PAM y CLARA - número inicial de cluster

Veamos cuál es el número óptimo de clústeres para `PAM`.

```
library(parameters)

set.seed(2137)
res_PAM <- n_clusters_silhouette(ordenadores,
                                 distance_method = "euclidean",
                                 clustering_function= cluster::pam,
                                 n_max = 5)



res_PAM
```

```
## The Silhouette method, based on the average quality of clustering, suggests that the optimal number o
```

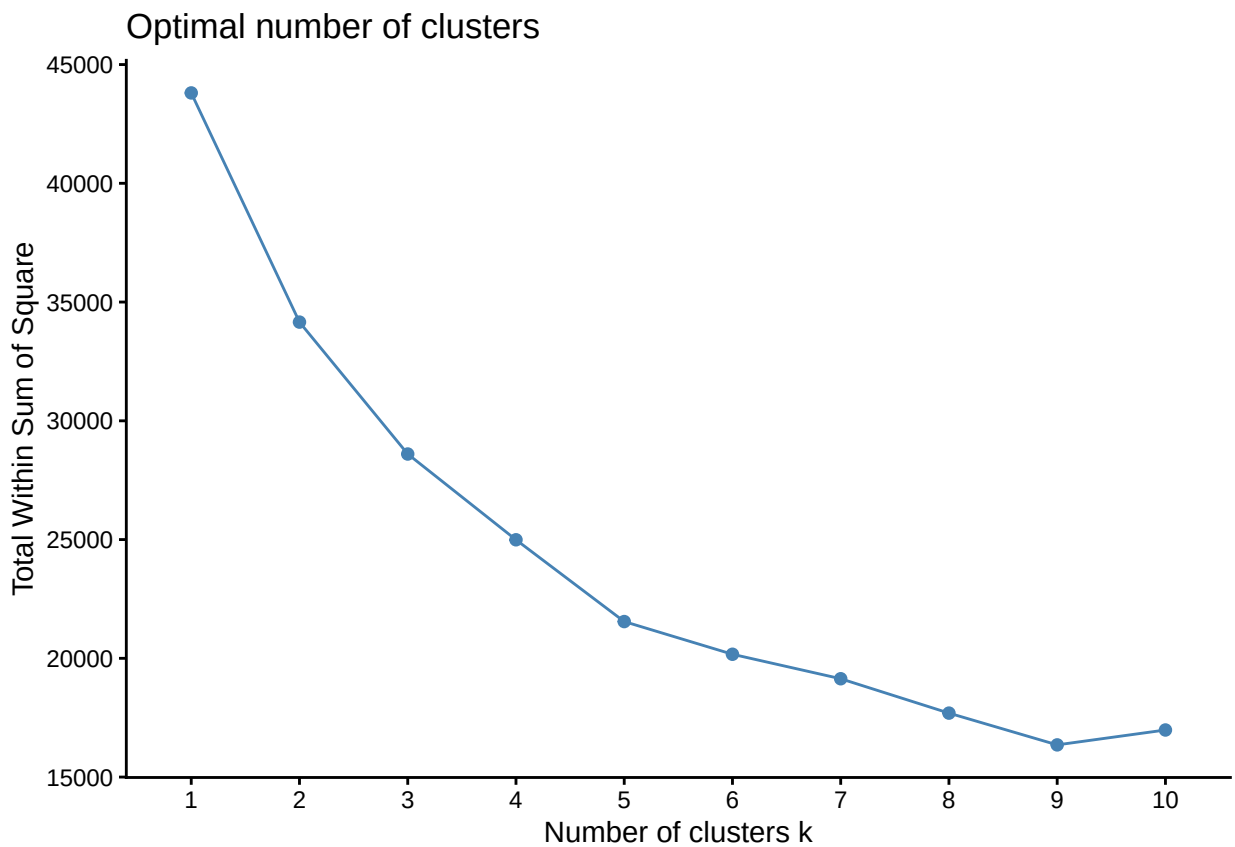
```
max(res_PAM$Silhouette)
```

```
## [1] 0.2472161
```

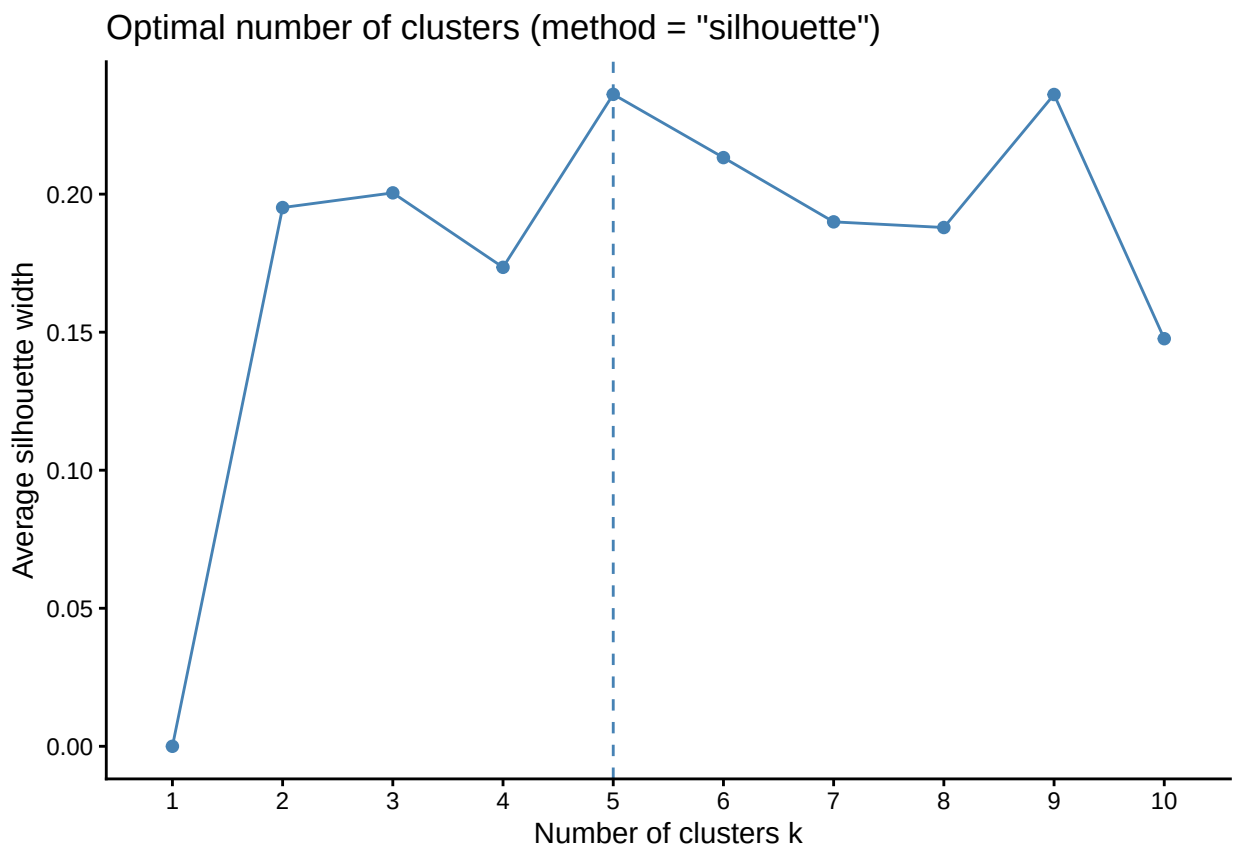
Ahora busquemos el número óptimo de clústeres para el método CLARA y comparemos si obtenemos resultados similares.

WSS y Silhouette

```r
set.seed(2137)
fviz_nbclust(ordenadores, cluster::clara, method = "wss") +
  theme_classic()
```




```r
fviz_nbclust(ordenadores, cluster::clara, method = "silhouette") +
  theme_classic()
```



Ambos métodos sugieren el uso de **5 clústeres**.

2.3. Algorítmo K-medias

En el primer bloque, se procede a la creación del modelo de agrupación:

`set.seed(2137)`: Se establece una semilla aleatoria para garantizar la reproducibilidad del análisis. Esto asegura que los resultados sean los mismos cada vez que se ejecute el código.

`kmeans(ordenadores, 6, nstart = 25)`: Se aplica el algoritmo de k-medias para dividir los datos en 6 clusters. El parámetro nstart = 25 indica que el algoritmo probará 25 configuraciones iniciales diferentes de los centroides, seleccionando la que minimice la variación total dentro de los grupos, lo que ayuda a encontrar un óptimo global.

```
set.seed(2137)
kmedias <- kmeans(ordenadores, 6, nstart = 25)
```

```
fviz_cluster(kmedias, data = ordenadores,
             geom = "point",
             pointsize = 0.5,
             alpha = 0.4,
             ellipse.type = "convex",
             ggtheme = theme_minimal())
```

Identificación de Grupos (6 Clusters):

- Cluster 6 (Magenta): Se sitúa en el extremo derecho de la gráfica. Es el grupo más disperso y diferenciado, lo que sugiere que representa ordenadores con características "extremas" o de gama muy alta que se alejan del promedio general.
- Clusters 2 (Amarillo) y 5 (Azul): Estos grupos están bien definidos en las zonas inferiores e izquierdas del gráfico. El Cluster 5 parece agrupar equipos con valores bajos en la dimensión 2, mientras que el Cluster 2 representa el segmento de entrada o más económico.

Solapamiento (Clusters 1, 3 y 4): Se observa una superposición significativa entre los clusters 1 (rojo), 3 (verde) y 4 (celeste) en la parte central. Esto indica que los ordenadores en estos grupos comparten características muy similares y que las fronteras entre estos segmentos no son tan nítidas en el espacio bidimensional.

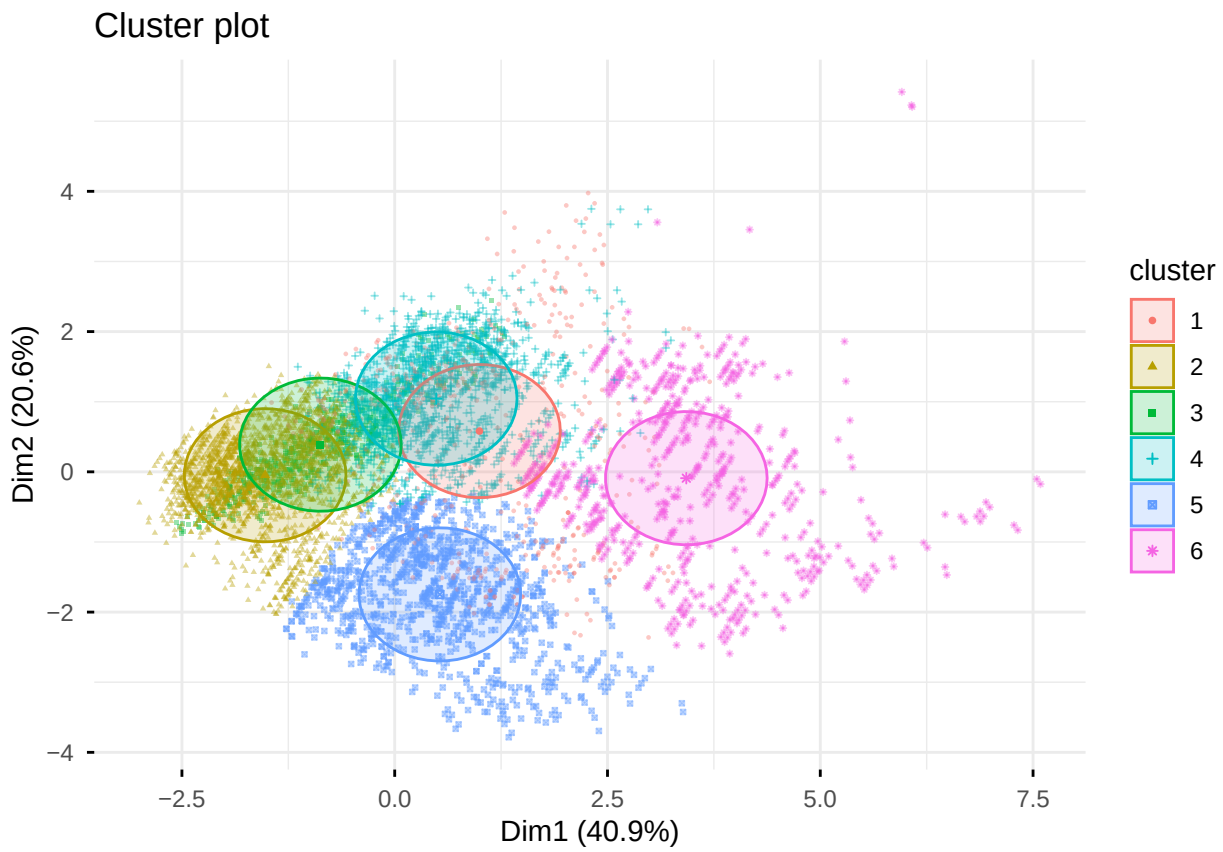
Morfología de los Clusters (Cascos Convexos):

Las líneas de colores representan los "convex hulls" (cascos convexos), que encierran todos los puntos pertenecientes a cada grupo. El hecho de que algunos cascos sean mucho más grandes que otros (como el del cluster 6 frente al 3) indica una mayor heterogeneidad dentro de esos grupos específicos.

Conclusión Visual:

Aunque el algoritmo ha logrado separar los datos en 6 categorías, la cercanía y el solapamiento entre los grupos centrales sugieren que el mercado de ordenadores es un continuo de características. El modelo es especialmente útil para identificar los nichos claramente diferenciados (como el cluster 6 y el cluster 5), mientras que los grupos centrales podrían requerir un análisis más profundo de sus centroides para distinguir sus ventajas competitivas específicas.

```r
fviz_cluster(kmedias,
             data=ordenadores,
             geom = "point",
             pointsize = 0.5,
             alpha = 0.4,
             ellipse.type = "euclid",
             ggtheme = theme_minimal()
)
```



Varianza Total Explicada:

El gráfico muestra que el Análisis de Componentes Principales (PCA) logra capturar un 61.5% de la varianza total de los datos en solo dos dimensiones (Dim1: 40.9% + Dim2: 20.6%). Al superar el 60%, podemos confiar en que la disposición de los puntos en el gráfico es una representación fiel de la estructura real del dataset de ordenadores.

Uso de Elipses Euclidianas (ellipse.type = "euclid"):

A diferencia del gráfico anterior (convexo), aquí se utilizan elipses euclidianas. Estas representan círculos de confianza basados en la distancia promedio desde el centroide:

- Permiten visualizar qué tan "compacto" es un grupo. Por ejemplo, los clusters 2 (amarillo) y 3 (verde) son muy compactos, lo que sugiere que los ordenadores en estos grupos tienen especificaciones técnicas muy similares entre sí.
- El Cluster 6 (magenta) tiene muchos puntos fuera de su elipse, lo que confirma que es el grupo con mayor cantidad de valores atípicos (outliers) o mayor diversidad de modelos.

Análisis de la Dimensión 1 (Eje X):

La Dim1 explica casi el 41% de la varianza. Se observa una clara progresión de izquierda a derecha. Es muy probable que esta dimensión represente la potencia o gama del ordenador (combinación de precio, RAM y procesador), situando a los modelos básicos a la izquierda y a los de alto rendimiento a la derecha.

Separación de los Grupos:

- Segmentación Vertical: El Cluster 5 (azul) se diferencia de los demás principalmente por la Dimensión 2, lo que sugiere que posee una característica única que lo separa del resto (podría ser, por ejemplo, el tamaño de pantalla o el tipo de disco duro).
- Solapamiento Central: Las elipses de los grupos 1 (rojo) y 4 (celeste) se cruzan significativamente, indicando que existe una zona de transición donde los ordenadores podrían pertenecer a cualquiera de los dos grupos según criterios muy sutiles.

Conclusión Técnica:

Esta visualización es ideal para identificar los "núcleos" de cada segmento de mercado. Mientras que los grupos 2 y 3 están muy consolidados, el grupo 6 representa un mercado mucho más heterogéneo y disperso.

```r
round(kmedias$centers,2)
```

```
##   price speed    hd   ram screen   ads trend
## 1  0.61  0.21  0.01 -0.06   2.64  0.02  0.17
## 2 -0.64 -0.66 -0.68 -0.67  -0.45  0.63 -0.34
## 3  0.24 -0.36 -0.70 -0.29  -0.39 -1.32 -1.55
## 4  0.87  0.45  0.22  0.49  -0.18  0.71 -0.29
## 5 -0.76  0.58  0.32 -0.29  -0.32 -0.88  1.21
## 6  0.87  0.55  2.09  2.13   0.55 -0.94  1.23
```

```r
round(aggregate(. ~ kmedias$cluster, data = ordenadores, FUN = mean)[, -1], 2)
```

```
##   price speed    hd   ram screen   ads trend
## 1  0.61  0.21  0.01 -0.06   2.64  0.02  0.17
## 2 -0.64 -0.66 -0.68 -0.67  -0.45  0.63 -0.34
## 3  0.24 -0.36 -0.70 -0.29  -0.39 -1.32 -1.55
## 4  0.87  0.45  0.22  0.49  -0.18  0.71 -0.29
## 5 -0.76  0.58  0.32 -0.29  -0.32 -0.88  1.21
## 6  0.87  0.55  2.09  2.13   0.55 -0.94  1.23
```

2.4. PAM

En esta sección, se aplica el algoritmo PAM, una técnica de particionamiento más robusta que K-means. A diferencia de K-means, que utiliza centroides (medias aritméticas), PAM utiliza medoides: objetos reales y representativos del conjunto de datos. Esta característica lo hace menos sensible a valores atípicos o outliers que puedan existir en el catálogo de ordenadores.

```r
pam.res <- pam(ordenadores, 5)
```

Análisis de la estructura de los grupos

`pam.res$clusinfo`: Este comando devuelve una tabla con información estadística crucial sobre los clusters formados. Incluye:

- `size`: El número de ordenadores asignados a cada grupo.
- `max_diss`: El diámetro del cluster (la distancia máxima entre objetos del mismo grupo).
- `av_diss`: La disimilitud media, que mide la cohesión interna.
- `separation`: La distancia al vecino más cercano, que mide qué tan bien separado está un grupo de otro.

```r
pam.res$clusinfo
```

```
##      size max_diss  av_diss diameter separation
## [1,]  619 3.724149 1.576137 5.849640 0.14453236
## [2,] 1728 4.868740 2.040813 7.830383 0.07058637
## [3,] 1998 3.668460 1.249144 4.919886 0.07058637
## [4,]  861 6.921456 2.354634 9.542334 0.10158333
## [5,] 1053 3.838234 1.616407 5.938265 0.10158333
```

```r
fviz_cluster(pam.res,
             geom = 'point',
             alpha = 0.6,
             pointsize = 0.5,
             ellipse.type = "t", # Concentration ellipse
             ggtheme = theme_minimal()
)
```

Este gráfico visualiza los resultados del algoritmo PAM utilizando 5 grupos. Las dos componentes principales explican un sólido 61.5% de la varianza total de los datos.

Elipses de Concentración: El uso de elipses tipo "t" permite identificar con claridad el núcleo central de cada categoría, ignorando visualmente los puntos más extremos para definir mejor cada segmento de mercado.

Distribución de los Grupos: Se observa una progresión clara a lo largo de la Dim1 (40.9%), donde el Cluster 4 (cian) representa los ordenadores de gama más alta y mayor potencia. Por el contrario, los Clusters 2 (amarillo) y 3 (verde) se agrupan en la zona de especificaciones más básicas.

Cohesión: Existe un solapamiento significativo entre los grupos 1, 2 y 3, lo que indica que estos segmentos

de ordenadores comparten características muy similares, mientras que el grupo 4 y el grupo 5 (morado) muestran una diferenciación más clara en sus perfiles técnicos.

2.5. CLARA

Para terminar, utilizaremos el método CLARA con los mismos parametros que para PAM, que facilita el procesamiento de grandes conjuntos de datos.

```
# Ustawiamy ziarno
set.seed(2137)

cluster_CLARA <- clara(ordenadores, k = 5,
                       stand = T, metric = "euclidean")
```

```
fviz_cluster(cluster_CLARA,
             geom = 'point',
             alpha = 0.6,
             pointsize = 0.5,
             ellipse.type = "t", # Concentration ellipse
             ggtheme = theme_minimal()
)
```

Conclusión del análisis con CLARA

Al aplicar el algoritmo CLARA sobre el conjunto de datos de ordenadores, se observa que los resultados obtenidos son prácticamente idénticos a los del algoritmo PAM. La distribución de los puntos, la formación de los 5 clusters y la varianza explicada por las componentes principales (61.5%) se mantienen constantes.

Puntos clave:

- Consistencia: El hecho de obtener los mismos resultados confirma la estabilidad de la estructura de grupos identificada previamente.
- Eficiencia: Aunque los resultados son similares, CLARA es el método más adecuado para este dataset de 6259 registros, ya que utiliza técnicas de muestreo para optimizar el tiempo de computación sin perder precisión en la identificación de los medoides.
- Segmentación: Se valida la existencia de un segmento diferenciado de alto rendimiento (Cluster 4) y una zona central donde los modelos de gama media y básica tienden a solaparse.

En resumen, CLARA confirma la validez del modelo PAM aportando una mayor eficiencia en el procesamiento de grandes volúmenes de información.